
Software Requirements Specification (SRS)

**Project: AI-based Car Fault Diagnosis Mobile Application
(CheckAR)**

Version: 1.0

Status: Draft

1. Introduction	1
1.1 Purpose	2
1.2 Scope	2
1.3 Definitions, Acronyms, and Abbreviations	2
1.4 References	2
1.5 Overview	3
2. Overall Description	3
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment	5
2.5 Design and Implementation Constraints.....	5
2.6 User Documentation	6
2.7 Assumptions and Dependencies	6
3. Specific Requirements	6
3.1 Functional Requirements	7
3.2 External Interface Requirements	8
3.2.1 User Interfaces	9
3.2.2 Hardware Interfaces	10
3.2.3 Software Interfaces	10
3.2.4 Communications Interfaces.....	10
3.3 Non-Functional Requirements	10
3.3.1 Performance Requirements	11
3.3.2 Reliability Requirements	11
3.3.3 Usability Requirements	11
3.3.4 Security Requirements	12
3.3.5 Maintainability and Portability.....	12
3.3.6 Quality Attributes	13
4. System Features.....	13
4.1 User Authentication System.....	14
4.1.1 Description.....	14
4.1.2 Functional Requirements	14
4.1.3 Technical Implementation	14
4.1.4 User Workflow	14
4.2 Dashboard Warning Light Scanner.....	14
4.2.1 Description.....	14
4.2.2 Functional Requirements	14
4.2.3 Technical Implementation	15
4.2.4 User Workflow	15

4.3 Engine Sound Analyzer	15
4.3.1 Description.....	15
4.3.2 Functional Requirements	15
4.3.3 Technical Implementation	15
4.3.4 User Workflow	15
4.4 Diagnosis and Reporting.....	15
4.4.1 Description.....	16
4.4.2 Functional Requirements	16
4.4.3 Technical Implementation	16
4.4.4 User Workflow	16
4.5 Maintenance Tracking.....	16
4.5.1 Description.....	16
4.5.2 Functional Requirements	16
4.5.3 Technical Implementation	16
4.5.4 User Workflow	17
4.6 Online / Offline Functionality	17
4.6.1 Description.....	17
4.6.2 Functional Requirements	17
4.6.3 Technical Implementation	17
4.6.4 User Workflow	17
5. Validation and Risk Mitigation	17
5.1 Stakeholder Engagement	18
5.2 Identified Risks and Mitigation Strategies.....	18
5.3 Testing Strategy	18
6. Appendices	19
A. Glossary.....	20
B. Analysis Models.....	20
C. Issue Tracking	20

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for CheckAR, an AI-based mobile application that enables car owners to diagnose vehicle faults using dashboard warning light recognition and engine sound analysis. It serves as the formal agreement between stakeholders and the development team and provides a basis for estimating costs, schedules, and validation criteria.

1.2 Scope

The application ("CheckAR") will allow users to:

- Register and log in with role-based access (car owners, mechanics, administrators)
- Capture and interpret dashboard warning lights via the device camera
- Record and classify engine sounds via the device microphone
- Receive automated explanations, repair suggestions, maintenance alerts, and video tutorials
- Track maintenance history and receive scheduled reminders
- Operate offline for core diagnostics and online for updated fault data and tutorials

The application will NOT:

- Replace professional diagnostic tools or mechanic expertise
- Interface directly with vehicle onboard computers (no OBD-II hardware connection)
- Provide real-time monitoring while the vehicle is in motion
- Guarantee 100% accuracy in all diagnostic scenarios

1.3 Definitions, Acronyms, and Abbreviations

- App: The CheckAR mobile application
- ML: Machine Learning
- UI/UX: User Interface / User Experience
- MVP: Minimum Viable Product
- OBD-II: On-Board Diagnostics II (vehicle self-diagnostic hardware standard)
- CV: Computer Vision
- PII: Personally Identifiable Information
- API: Application Programming Interface
- 2FA: Two-Factor Authentication
- RBAC: Role-Based Access Control
- MFCC: Mel-Frequency Cepstral Coefficients (audio feature extraction technique)

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- CheckAR_Task2_DataGathering.docx: Task 2 survey data gathering and cleaning report

- CheckAR_Requirements_Analysis_Report.docx: Task 3 requirements analysis and prioritization
- ISO/IEC 25010:2011: Systems and Software Quality Requirements and Evaluation (SQuaRE)
- ISO 26262: Road Vehicles — Functional Safety Standard
- OWASP Mobile Application Security Verification Standard (MASVS)

1.5 Overview

The remainder of this document covers:

- Section 2: Overall description of the product, including perspective, functions, user characteristics, operating environment, constraints, and assumptions
- Section 3: Specific requirements, including functional, external interface, and non-functional requirements
- Section 4: System features with detailed descriptions of each core module
- Section 5: Validation methods, testing strategy, and risk mitigation
- Section 6: Appendices including glossary, analysis models, and issue tracking

2. Overall Description

2.1 Product Perspective

CheckAR is a standalone mobile application designed for Android and iOS platforms using the Flutter cross-platform framework. It leverages on-device ML models (TensorFlow Lite) for image and audio classification, with optional cloud connectivity for data updates and enhanced features.

The application relates to existing products as follows:

- Unlike professional OBD-II scanners, CheckAR requires no physical connection to the vehicle
- Compared to existing automotive apps, it uniquely combines visual dashboard scanning and engine audio analysis in a single tool
- Integrates with third-party content platforms (YouTube Data API) for educational repair materials

2.2 Product Functions

- User Authentication: Registration, login, and profile management with role-based access control
- Dashboard Light Scanner: Detect and classify multiple warning lights from a single captured image
- Engine Sound Analyzer: Record, preprocess, and classify engine noises (knocking, squealing, hissing, grinding, clicking)
- Diagnosis Report: Provide fault cause, urgency level, repair tips, and relevant video tutorial links
- Maintenance Tracker: Log past diagnostics, associated costs, and schedule automated reminders
- Knowledge Base: Access to common car problems and contextual repair solutions
- Offline Mode: All core diagnostic features without internet; Online Mode: Fetch updated models, fault databases, and tutorials

2.3 User Classes and Characteristics

Primary: Car Owners (Non-Technical)

- Demographics: Ages 18–65, varying technical expertise
- Goals: Quick diagnosis, basic understanding of car issues without specialist knowledge
- Technical expertise: Limited automotive knowledge
- Usage frequency: Occasional, primarily when warning lights appear or unusual sounds are detected
- Access level: Standard diagnostic and maintenance features

Secondary: Mechanics / Technicians

- Goals: Quick pre-checks and a client education tool during service appointments
- Technical expertise: Professional automotive knowledge
- Usage frequency: Regular, across multiple vehicles

- Access level: Enhanced diagnostic features and ability to add professional notes to diagnoses

Tertiary: Content Creators

- Goals: Provide and manage tutorial content linked from diagnostic results
- Technical expertise: Automotive expertise and content production skills
- Usage frequency: Content submission and periodic updates
- Access level: Content management interface

System Administrators

- Goals: Manage backend services, update ML models, monitor performance metrics
- Technical expertise: IT and ML engineering expertise
- Usage frequency: Regular maintenance and monitoring
- Access level: Full system access, user management, and analytics dashboard

2.4 Operating Environment

Mobile Devices:

- Android 8.0 (API level 26) or higher
- iOS 12.0 or higher
- Minimum 2 GB RAM, 100 MB free storage
- Rear-facing camera with minimum 5 MP resolution
- Device microphone (noise cancellation preferred)

Network:

- Internet connectivity (Wi-Fi or cellular) required for online features
- Full offline capability for all core diagnostic features

Backend Services:

- Cloud-based user authentication (Firebase Authentication)
- Cloud-based ML model training and update infrastructure
- Content delivery network (CDN) for tutorial metadata and model packages
- Secure database for user profiles and preferences

2.5 Design and Implementation Constraints

Technical Constraints:

- On-device ML model size must not exceed 10 MB
- TensorFlow Lite required for all on-device ML inference
- Flutter framework for cross-platform development (Android and iOS from a single codebase)
- Diagnostic feedback latency must not exceed 5 seconds
- Battery consumption must not exceed 5% per full diagnostic session

Business Constraints:

- No personal data stored beyond the scope of the app's stated purpose
- No advertisements displayed in any diagnostic or primary application view

- Initial app download size not to exceed 50 MB

Regulatory Constraints:

- Must include a clear disclaimer regarding diagnostic accuracy limitations on all result screens
- Must comply with applicable data protection regulations (GDPR, CCPA)
- Cannot claim to replace professional mechanical or electronic diagnosis
- Authentication system must comply with industry security standards

2.6 User Documentation

- In-app interactive tutorial for first-time users
- FAQ section covering common use cases and disclaimer information
- Online help center with troubleshooting guides
- Video demonstrations for all key features
- Contextual tooltips on interface elements
- Role-specific guidance for car owners, mechanics, and administrators

2.7 Assumptions and Dependencies

- Users grant camera and microphone permissions at runtime
- Access to YouTube Data API for tutorial video retrieval and linking is maintained
- Sufficient labeled audio and image datasets are available for ML model training
- Users possess basic smartphone operation familiarity
- Adequate ambient lighting exists for dashboard photo capture
- Vehicle engine is accessible and audible for sound recording
- Firebase authentication service remains available at 99.9% uptime

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement	Priority	Source
FR1	The app shall provide user registration functionality with email verification.	Must-Have	Security Standards
FR2	The app shall support role-based user authentication (car owner, mechanic, and administrator).	Must-Have	Project Design
FR3	The app shall allow users to recover forgotten passwords securely via email.	Must-Have	Security Standards
FR4	The app shall maintain user sessions with appropriate inactivity timeout controls.	Must-Have	Security Standards
FR5	The app shall allow users to capture dashboard images and detect warning light icons.	Must-Have	Survey, Interviews
FR6	The app shall interpret each detected icon and display its meaning and urgency level.	Must-Have	Survey, Expert Interviews
FR7	The app shall record engine sound clips up to 10 seconds using the device microphone.	Must-Have	Survey, Interviews
FR8	The app shall classify engine sounds into predefined fault categories (e.g., knocking, squealing, hissing).	Must-Have	Requirements Analysis
FR9	The app shall provide plain-language explanations and repair suggestions for detected faults.	Must-Have	Survey, Interviews
FR10	The app shall include links to relevant YouTube video tutorials for each diagnosed issue.	Should-Have	Survey, User Feedback
FR11	The app shall store past diagnostics locally with timestamps and results.	Should-Have	Survey, Interviews
FR12	The app shall send maintenance reminders based on user-defined schedules or fault urgency.	Should-Have	Requirements Analysis
FR13	The app shall operate offline for image and audio diagnostics using preloaded on-device models.	Must-Have	Survey, Interviews
FR14	The app shall fetch updated fault databases and tutorial metadata when internet is available.	Should-Have	Survey, Interviews
FR15	The app shall allow users to input vehicle make, model, and year for more specific diagnostics.	Should-Have	Expert Interviews
FR16	The app shall detect multiple warning lights simultaneously in a single captured image.	Must-Have	Project Description
FR17	The app shall estimate maintenance urgency on a three-tier scale (Immediate, Soon, Monitor).	Must-Have	Project Description
FR18	The app shall allow users to save and share diagnostic reports.	Could-Have	User Feedback
FR19	The app shall provide preventive maintenance tips specific to identified issues.	Should-Have	Expert Interviews
FR20	The app shall enable mechanics to add professional notes to diagnoses (mechanic role only).	Should-Have	Project Design

ID	Requirement	Priority	Source
FR21	The app shall provide administrators with full user management capabilities.	Must-Have	Project Design
FR22	The app shall allow users to manage their profile information and notification preferences.	Should-Have	User Feedback

3.2 External Interface Requirements

3.2.1 User Interfaces

Authentication Screens:

- Registration form with email and password fields, role selection, and email verification step
- Login screen with email/password input and 'Forgot Password' link
- Password recovery screen with email confirmation flow
- Profile management screen for personal info, vehicle details, and preferences

Home Screen:

- Primary action buttons: 'Scan Dashboard' and 'Record Engine Sound'
- Quick navigation to diagnostic history and settings
- Active vehicle profile display with role-specific feature visibility

Dashboard Scanner Screen:

- Camera viewfinder with framing guide overlay
- Capture button and flash toggle for low-light conditions
- Preview with confirm or retake options before processing

Sound Recorder Screen:

- Recording button with countdown timer display
- Real-time audio level visualization waveform
- On-screen positioning instructions for optimal recording

Results Screen:

- Detected warning icons displayed with labels and urgency color coding
- Audio classification results with confidence indicators
- Expandable detail panels: explanation, probable causes, urgency, and tutorial link
- Save and share options; professional notes section visible to mechanics only

History Screen:

- Chronological list of past diagnostics with date, time, and issue summary
- Filter by date range, issue type, or urgency level
- Option to compare multiple past diagnoses

Admin Dashboard:

- User management interface (add, remove, edit, assign roles)
- System performance metrics and diagnostic session statistics
- ML model update controls and deployment management

Settings Screen:

- Notification and reminder preferences

- Data usage controls (Wi-Fi only downloads)
- Vehicle profile management
- Authentication and security settings

3.2.2 Hardware Interfaces

- Camera: Rear-facing with autofocus, flash control for low-light, minimum 5 MP resolution
- Microphone: Device microphone access, support for external microphones, 44.1 kHz sampling at 16-bit depth
- Storage: Local storage for diagnostic history and cached model data, secure storage for authentication tokens
- Display: Support for multiple screen sizes and resolutions, portrait and landscape orientation, accessible contrast and text sizing

3.2.3 Software Interfaces

- Authentication Service: Firebase Authentication; OAuth 2.0 integration (optional social login); token-based session management
- TensorFlow Lite: On-device ML inference using versioned .tflite models; model update and version tracking
- YouTube Data API: Tutorial video retrieval and embedding; fault-specific video search
- Local Database: SQLite for diagnostic history storage; secure credential storage; JSON files for vehicle specifications
- Push Notification Service: Firebase Cloud Messaging for maintenance reminders and diagnostic alerts

3.2.4 Communications Interfaces

- HTTPS/TLS for all secure data fetches and API calls
- REST API for backend services and model update delivery
- Secure download of ML model update packages with checksum verification
- Encrypted upload of anonymized diagnostic feedback data (with explicit user consent only)

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

Response Time:

- Maximum 5 seconds for image (dashboard) classification
- Maximum 7 seconds for audio (engine sound) classification
- Maximum 2 seconds for all UI transitions
- Authentication processes complete within 3 seconds

Startup Time:

- App launches within 5 seconds on all supported devices
- On-device ML models load within 5 seconds after launch
- Authentication state verification within 2 seconds

Throughput:

- Support up to 50 diagnostic sessions per day per user
- Database queries complete in under 100 ms
- Handle 100+ concurrent authentication requests

Resource Utilization:

- Memory usage below 200 MB during operation
- CPU usage below 30% during active analysis
- Battery consumption below 1% during standby

3.3.2 Reliability Requirements

Availability:

- 99% uptime for all online features
- 100% availability for all offline core diagnostic features
- Authentication services available 99.9% of the time

MTBF (Mean Time Between Failures):

- Fewer than 5 crashes per 1000 user sessions
- Authentication system failure rate below 0.5%

Error Handling:

- Graceful degradation message if camera or microphone access is denied
- Appropriate fallback behaviour for all network failure scenarios
- User-friendly error messages with actionable recovery options
- Clear guidance on authentication failure with retry logic

Recovery:

- Auto-save of partial diagnostic data during system interruptions
- Automatic retry for failed network operations with exponential back-off
- Session recovery after unexpected application termination

3.3.3 Usability Requirements

UI Complexity:

- Designed for users with no technical or automotive background
- Maximum 3 taps from the home screen to reach any primary function
- Consistent design language and navigation patterns throughout
- Clear role-specific interface elements for car owners, mechanics, and admins

Learnability:

- First-time users can complete registration in under 2 minutes
- First-time users can complete a full diagnosis in under 3 minutes
- Interactive onboarding tutorial for all new users
- Role-specific contextual guidance for each user type

Accessibility:

- Full support for screen readers and dynamically scaled fonts
- Color schemes with adequate contrast ratios throughout
- Voice commands available for all key primary functions
- Compliance with WCAG 2.1 AA accessibility standards

3.3.4 Security Requirements

Authentication:

- Multi-factor authentication option available for all users
- Password complexity requirements enforced at registration and change
- Account lockout after multiple consecutive failed login attempts
- Secure credential storage using industry-standard encryption (AES-256)

Data Privacy:

- Audio and image data processed entirely on-device; no PII transmitted to external servers
- Anonymized usage analytics shared only with explicit, informed user opt-in
- Secure local storage for all diagnostic history
- User data segregation enforced across all system roles

Permissions:

- Explicit runtime permission requests for camera and microphone access
- Granular per-permission management available in settings
- Clear user-facing explanation of why each permission is required
- Role-Based Access Control (RBAC) enforced for all application features

Data Integrity:

- Checksum verification for all downloaded ML model updates
- Prevention of unauthorized modification of diagnostic history records
- Audit trails maintained for all administrative actions

3.3.5 Maintainability and Portability

Codebase Structure:

- Modular Flutter codebase with strict separation of UI, business logic, and data layers
- Comprehensive inline code documentation and architecture decision records
- Authentication module fully isolated for independent updates

Platform Support:

- Identical full feature sets on Android and iOS
- Tablet-optimized layouts for larger screen sizes
- Consistent authentication experience across all supported platforms

Upgradability:

- Over-the-air (OTA) ML model updates with incremental delivery to minimize data usage
- Versioned database migration strategy for all app updates
- Authentication system upgradable without disruption to existing user sessions

3.3.6 Quality Attributes**Accuracy:**

- At least 90% diagnostic accuracy for common dashboard warning lights
- At least 85% accuracy for engine sound classification
- False positive rate below 5% for all diagnostic categories
- Zero false positives in the authentication system

Efficiency:

- Total end-to-end diagnosis response time at or below 5 seconds
- Battery consumption below 1% per complete diagnostic session
- Data usage below 1 MB per online session (excluding video playback)

Privacy:

- No tracking of user location or behavioral patterns
- All user data anonymized before any external transmission
- Full compliance with GDPR, CCPA, and applicable local data protection regulations

Data Usage:

- Optimized for low-bandwidth environments (target: functional at sub-2 Mbps)
- Compressed model update packages with delta-only delivery where possible
- Wi-Fi-only option available for all large downloads

4. System Features

4.1 User Authentication System

4.1.1 Description

The user authentication system provides secure registration, login, and role-based access control for car owners, mechanics, and administrators, ensuring that each user type accesses only the features and data appropriate to their role.

4.1.2 Functional Requirements

- FR1: User registration with email verification
- FR2: Role-based authentication
- FR3: Password recovery
- FR4: Session management
- FR21: Administrator user management
- FR22: Profile management

4.1.3 Technical Implementation

- Firebase Authentication or equivalent cloud backend service
- Token-based session management with configurable inactivity expiration
- Secure credential storage using AES-256 encryption
- Role-Based Access Control (RBAC) enforced at both client and server layers
- Email verification workflow with resend capability
- Password policy enforcement (minimum 8 characters, complexity requirements)

4.1.4 User Workflow

- Registration: User selects 'Register'; provides email, password, and selects role (car owner or mechanic); verification email sent; user confirms email; completes profile setup.
- Login: User enters credentials; system authenticates and determines role; user directed to role-specific home screen.
- Password Recovery: User selects 'Forgot Password'; provides registered email; receives reset link; creates new password.
- Profile Management: User accesses profile from settings; updates personal information, active vehicle details, and notification preferences; changes saved securely.

4.2 Dashboard Warning Light Scanner

4.2.1 Description

The dashboard warning light scanner allows users to photograph their vehicle's instrument cluster and automatically identifies and interprets any active warning lights present in the image.

4.2.2 Functional Requirements

- FR5: Image capture functionality
- FR6: Warning light interpretation with urgency labelling
- FR16: Multi-light simultaneous detection

4.2.3 Technical Implementation

- Computer vision model using CNN architecture trained on dashboard warning light datasets
- Image preprocessing pipeline for enhancement across varied lighting conditions
- On-device database of 100+ common warning light symbols across major vehicle manufacturers
- Real-time bounding-box highlighting of all detected symbols in the captured image

4.2.4 User Workflow

- User selects 'Scan Dashboard' from the home screen
- Camera viewfinder appears with framing guide overlay
- User captures the dashboard image; app processes and highlights all detected warning lights
- User confirms image or selects retake; app displays detailed information for each detected light

4.3 Engine Sound Analyzer

4.3.1 Description

The engine sound analyzer records audio from the vehicle engine using the device microphone and applies machine learning to classify sounds associated with common mechanical faults.

4.3.2 Functional Requirements

- FR7: Audio recording capability
- FR8: Engine sound classification into predefined fault categories

4.3.3 Technical Implementation

- Audio preprocessing pipeline including ambient noise reduction
- Feature extraction using MFCC (Mel-Frequency Cepstral Coefficients)
- Classification model using CNN or LSTM architecture
- Detection of at least 10 common problematic engine sound categories

4.3.4 User Workflow

- User selects 'Record Engine Sound' from the home screen
- App provides on-screen positioning instructions for optimal microphone placement
- User starts recording (5–10 seconds); app preprocesses and classifies the audio
- Results display with confidence scores and fault descriptions for each detected category

4.4 Diagnosis and Reporting

4.4.1 Description

The diagnosis system combines results from warning light and sound analysis to produce comprehensive fault information, urgency ratings, repair suggestions, and linked educational tutorial content.

4.4.2 Functional Requirements

- FR9: Plain-language explanation and repair suggestions
- FR10: Video tutorial links from YouTube
- FR17: Three-tier urgency estimation (Immediate, Soon, Monitor)
- FR19: Preventive maintenance tips
- FR20: Professional notes for mechanic role

4.4.3 Technical Implementation

- Rule-based fusion system for combining visual and audio diagnostic outputs
- Urgency scoring algorithm based on safety implications and potential component damage risk
- Curated knowledge base of fault-specific repair suggestions and maintenance tips
- YouTube Data API integration for contextually relevant tutorial video retrieval
- Role-gated notes system with mechanic-only write access and owner-visible read access

4.4.4 User Workflow

- After scan or recording, the results screen displays all identified issues
- Each issue card shows: description, probable causes, urgency level (colour-coded), and tutorial link
- User can expand any issue card for full detail view
- Mechanics can add professional notes; car owners see a read-only notes section
- User can save report to history or share externally with a mechanic

4.5 Maintenance Tracking

4.5.1 Description

The maintenance tracking system stores all diagnostic history locally and sends scheduled reminders for regular maintenance tasks and follow-up actions on previously detected issues.

4.5.2 Functional Requirements

- FR11: Diagnostic history storage with timestamps
- FR12: Maintenance reminder notifications
- FR18: Diagnostic report saving and sharing

4.5.3 Technical Implementation

- Local SQLite database for all diagnostic history; cloud synchronization for registered users
- Firebase Cloud Messaging for scheduled maintenance reminder delivery
- PDF export for diagnostic reports; secure sharing via OS-native share sheet

4.5.4 User Workflow

- User accesses history from the home screen bottom navigation
- Chronological list of all past diagnostics displayed with issue type, urgency badge, and date
- User can filter by date range, issue category, or urgency level
- Detailed view available for each past diagnosis; options to set a follow-up reminder or share

4.6 Online / Offline Functionality

4.6.1 Description

CheckAR delivers all core diagnostic functionality without any internet connection, while enabling enhanced features and model and database updates when connectivity is available.

4.6.2 Functional Requirements

- FR13: Fully offline operation for core image and audio diagnostics
- FR14: Online fetch of updated fault databases and tutorial metadata
- FR15: Vehicle-specific diagnostic refinement using make/model/year input

4.6.3 Technical Implementation

- TensorFlow Lite models bundled within the app package for fully offline operation
- Incremental (delta) model updates to minimize cellular data consumption
- Vehicle make/model/year database with common fault associations stored locally
- Version management system for ML models and fault databases
- Offline authentication state caching for previously authenticated users

4.6.4 User Workflow

- App operates with full core capabilities in offline mode for all authenticated users
- Background connectivity checks run passively when network is restored
- User receives a notification of available updates including package size; can defer to Wi-Fi
- Incremental updates download silently in the background during Wi-Fi connections

5. Validation and Risk Mitigation

5.1 Stakeholder Engagement

The requirements for CheckAR were gathered and validated through:

- Structured surveys targeting automotive mechanics of varying experience levels, conducted as part of Task 2 (Data Gathering and Cleaning)
- Interviews with car owners representing the primary non-technical user group
- Review sessions with project stakeholders and the course instructor
- Comparative analysis of existing car diagnostic mobile applications

5.2 Identified Risks and Mitigation Strategies

Risk	Impact	Likelihood	Mitigation Strategy
Authentication system breach	Critical	Low	Implement OAuth 2.0, 2FA, and AES-256 storage; schedule regular penetration testing
Inaccurate fault classification	High	Medium	Iterative ML model retraining using mechanic survey data and real-world test cases
Limited labeled dataset availability	Medium	High	Use public automotive datasets; crowd-source anonymized data with explicit user consent
Audio interference in real-world recordings	Medium	Medium	Mandatory ambient noise reduction preprocessing pipeline in the audio classification module
On-device ML model exceeds 10 MB size limit	High	Low	Apply model quantization and weight pruning; benchmark size continuously during development
Low adoption due to interface complexity	High	Medium	Prioritize UI/UX simplicity; conduct usability testing with representative non-technical users
Battery drain during extended diagnostic use	Medium	Medium	Optimize camera and microphone scheduling; implement power-saving idle and standby modes
False sense of security from app diagnosis	High	Medium	Prominent accuracy disclaimers on all result screens; urgency indicators direct users to professionals
Privacy concerns with audio and vehicle data	Medium	Medium	All processing done on-device; explicit opt-in consent for any anonymized data sharing
User resistance to mandatory registration	Medium	High	Offer a limited guest mode allowing basic diagnostic features without account creation

5.3 Testing Strategy

- Security Testing: Penetration testing and vulnerability assessment of the authentication system and all API endpoints
- Unit Testing: Individual components — authentication module, camera pipeline, audio recording, ML classification models
- Integration Testing: End-to-end combined diagnostic workflows from image/audio capture through to report output
- Performance Testing: Response time benchmarks, resource utilization, and battery consumption profiling

- Usability Testing: Representative test groups across non-technical car owners and mechanics
- Beta Testing: Staged limited release to gather real-world feedback before full public deployment
- Model Validation: Accuracy, precision, recall, and false positive rate evaluation against held-out labeled datasets

6. Appendices

A. Glossary

- **Authentication:** The process of verifying a user's identity before granting system access
- **RBAC:** Role-Based Access Control — restricting system access based on assigned user roles
- **Diagnosis:** The process of identifying mechanical or electrical faults in a vehicle
- **Model Inference:** Running a trained ML model on new input data to generate predictions
- **Warning Light:** A dashboard indicator lamp signalling the operational status of a vehicle system
- **OBD System:** The On-Board Diagnostics system built into modern vehicles
- **Engine Knocking:** An abnormal combustion sound indicative of potential engine damage
- **False Positive:** An incorrect identification of a non-existent fault or warning
- **Model Quantization:** A technique to reduce ML model file size for efficient mobile deployment
- **MFCC:** Mel-Frequency Cepstral Coefficients — audio features used in engine sound classification
- **Token:** A digital credential representing authentication authorization within a user session

B. Analysis Models

- **Use Case Diagram:** Major use cases — Register, Login, ScanDashboard, RecordEngineSound, ViewResults, ViewHistory
- **Data Flow Diagram:** Flow — Authentication → Input Capture → Preprocessing → ML Inference → Diagnosis Report Output
- **Class Diagram:** Relationships among core components — User, Vehicle, DiagnosisReport, MaintenanceRecord, AdminDashboard
- **Sequence Diagram:** Detailed interaction flows during authentication and the full diagnostic workflow
- **State Diagram:** Application state transitions across a full user session lifecycle

C. Issue Tracking

ID	Issue	Status	Resolution
ISSUE-001	Authentication token expiration handling	In Progress	Implementing refresh token mechanism with silent re-authentication
ISSUE-002	Model accuracy in low-light dashboard capture	Open	Investigating histogram equalization and denoising in preprocessing
ISSUE-003	Audio classification accuracy with background noise	In Progress	Adding ambient noise isolation algorithms to the preprocessing pipeline
ISSUE-004	Battery consumption during extended scanning sessions	Resolved	Optimized camera frame rate and CPU scheduling during capture
ISSUE-005	Support for international and regional warning light symbols	Open	Expanding training dataset with international vehicle manufacturer databases

ID	Issue	Status	Resolution
ISSUE-006	Role transition approval workflow (car owner to mechanic upgrade)	Open	Designing admin-gated approval workflow and professional verification process
ISSUE-007	Tutorial relevance accuracy for specific diagnosed faults	In Progress	Refining YouTube API metadata matching algorithm using structured fault taxonomy